

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

- First Line Contains Integer m – Size of array
- Next m lines Contains m numbers – Elements of an array

Output Format

- First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

Ace editor not ready. Perhaps reload page?

Falling back to raw text area.

```
#include <stdio.h>

int countZeroes(int arr[], int low, int high, int n) {
    if (high >= low) {
        int mid = low + (high - low) / 2;
        if ((mid == 0 || arr[mid - 1] == 1) && arr[mid] == 0)
            return n - mid;
        if (arr[mid] == 1)
            return countZeroes(arr, mid + 1, high, n);
        return countZeroes(arr, low, mid - 1, n);
    }
    return 0;
}

int main() {
    int m;
    if (scanf("%d", &m) != 1) return 0;
    int arr[m];
    for (int i = 0; i < m; i++) scanf("%d", &arr[i]);
    printf("%d\n", countZeroes(arr, 0, m - 1, m));
}
```

	Input	Expected	Got	
✓	5 1 1 1 0 0	2	2	✓
✓	10 1 1 1 1 1 1 1 1 1 1 1	0	0	✓

✓	8 0 0 0 0 0 0 0 0	8	8	✓
---	---	---	---	---

✓	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	✓
---	---	---	---	---

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: `nums = [3,2,3]`

Output: `3`

Example 2:

Input: `nums = [2,2,1,1,1,2,2]`

Output: `2`

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 104`
- `-231 <= nums[i] <= 231 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int findMajorityElement(int nums[], int n) {
3     int candidate = 0, count = 0;
4     for (int i = 0; i < n; i++) {
5         if (count == 0) {
6             candidate = nums[i];
7         }
8         count += (nums[i] == candidate) ? 1 : -1;
9     }
10    return candidate;
11 }
12 int main() {
13     int n;
14     if (scanf("%d", &n) != 1) return 0;
15     int nums[n];
16     for (int i = 0; i < n; i++) {
17         scanf("%d", &nums[i]);
18     }
19     printf("%d\n", findMajorityElement(nums, n));
20     return 0;
21 }
```

	Input	Expected	Got	
✓	3 3 2 3	3	3	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

- First Line Contains Integer n – Size of array
- Next n lines Contains n numbers – Elements of an array
- Last Line Contains Integer x – Value for x

Output Format

- First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 int findFloor(int arr[], int low, int high, int x) {
3     if (low > high) return -1;
4     if (x >= arr[high]) return arr[high];
5     int mid = low + (high - low) / 2;
6     if (arr[mid] == x || (mid > 0 && arr[mid - 1] <= x && x < arr[mid]))
7         return arr[mid] == x ? arr[mid] : arr[mid - 1];
8     if (x < arr[mid])
9         return findFloor(arr, low, mid - 1, x);
10    return findFloor(arr, mid + 1, high, x);
11 }
12 int main() {
13     int n, x;
14     if (scanf("%d", &n) != 1) return 0;
15     int arr[n];
16     for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
17     if (scanf("%d", &x) != 1) return 0;
18     printf("%d\n", findFloor(arr, 0, n - 1, x));
19     return 0;
20 }
```

	Input	Expected	Got	
✓	6 1 2 8 10 12 19 5	2	2	✓
✓	5 10 22 85 108 129 100	85	85	✓
✓	7 3 5 7 9 11 13 15 10	9	9	✓

Passed all tests! ✓

Correct

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

- First Line Contains Integer n – Size of array
- Next n lines Contains n numbers – Elements of an array
- Last Line Contains Integer x – Sum Value

Output Format

- First Line Contains Integer – Element1
- Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 void findPair(int arr[], int l, int r, int x) {
3     if (l >= r) { printf("No\n"); return; }
4     int sum = arr[l] + arr[r];
5     if (sum == x) { printf("%d\n%d\n", arr[l], arr[r]); return; }
6     if (sum < x) findPair(arr, l + 1, r, x);
7     else findPair(arr, l, r - 1, x);
8 }
9 int main() {
10     int n, x;
11     if (scanf("%d", &n) != 1) return 0;
12     int arr[n];
13     for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
14     if (scanf("%d", &x) != 1) return 0;
15     findPair(arr, 0, n - 1, x);
16     return 0;
17 }
```

	Input	Expected	Got	
✓	4	4	4	✓
	2	10	10	
	4			
	8			
	10			
	14			
✓	5	No	No	✓
	2			
	4			
	6			
	8			
	10			
	100			

Passed all tests! ✓

Correct

Write a Program to Implement the Quick Sort Algorithm

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

For example:

Input	Result
5 12 34 67 78 98	12 34 67 78 98

Answer:

```
1 #include <stdio.h>
2 void swap(int *a, int *b) {
3     int t = *a; *a = *b; *b = t;
4 }
5 int partition(int arr[], int low, int high) {
6     int pivot = arr[high], i = low - 1;
7     for (int j = low; j < high; j++) {
8         if (arr[j] < pivot) {
9             i++;
10            swap(&arr[i], &arr[j]);
11        }
12    }
13    swap(&arr[i + 1], &arr[high]);
14    return i + 1;
15 }
16 void quickSort(int arr[], int low, int high) {
17     if (low < high) {
18         int pi = partition(arr, low, high);
19         quickSort(arr, low, pi - 1);
20         quickSort(arr, pi + 1, high);
21     }
22 }
23 int main() {
24     int n;
25     if (scanf("%d", &n) != 1) return 0;
26     int arr[n];
27     for (int i = 0; i < n; i++) scanf("%d", &arr[i]);
28     quickSort(arr, 0, n - 1);
29     for (int i = 0; i < n; i++)
30         printf("%d%c", arr[i], (i == n - 1) ? '\n' : ' ');
31     return 0;
32 }
```

	Input	Expected	Got	
✓	5 12 34 67 78 98	12 34 67 78 98	12 34 67 78 98	✓
✓	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✓
✓	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✓

Passed all tests! ✓